

NBSAP MONITORING SYSTEM

Comprehensive System Design and Requirements Document

June 2026

Contents

0.1	Document Control	3
0.1.1	Revision History	3
0.1.2	Purpose of This Document	3
1	Executive Summary	4
2	1. Introduction and Background	5
2.1	1.1 Background: NBSAP and the Kunming-Montreal Global Biodiversity Framework	5
2.2	1.2 Purpose of the System	5
2.3	1.3 Scope of This Document	5
2.4	1.4 Intended Audience	6
2.5	1.5 Verification Methodology	6
3	2. Business Process Document	7
3.1	2.1 AS-IS Process (Pre-Digitisation Baseline)	7
3.2	2.2 TO-BE Process (As Implemented)	7
3.3	2.3 Reporting Cycle	8
3.4	2.4 Role-Change Approval Process	8
4	3. Software Requirements Specification	9
4.1	3.1 Functional Requirements	9
4.2	3.2 Non-Functional Requirements	10
5	4. User Roles and Permissions	13
5.1	4.1 Role Definitions	13
5.2	4.2 Permission Matrix	13
5.3	4.3 Route-Level Access Enforcement	14
6	5. Business Rules	15
7	6. Data Requirements and Data Dictionary	17
7.1	6.1 Core Entities	17
7.2	6.2 Indicator and Target Volume	18
8	7. Use Case Specifications	19
8.1	UC-001: User Authentication	19
8.2	UC-002: Submit a Progress Report	19
8.3	UC-003: Verify a Pending Report	19
8.4	UC-004: View National Progress Dashboard	19
8.5	UC-005: View the Interactive Map	20
8.6	UC-006: View Biodiversity Data (RBIS)	20
8.7	UC-007: Export a Report to PDF	20
8.8	UC-008: Request a Role Change	20

8.9	UC-009: Administer Targets, Indicators, and Programmes	20
9	8. System Behaviour and State Models	21
9.1	8.1 Report Status Lifecycle	21
9.2	8.2 Role-Change Request Lifecycle	21
9.3	8.3 User Session State	21
10	9. Validation Rules	22
11	10. Error Handling and Exception Management	23
12	11. System Architecture	24
12.1	11.1 Technology Stack (Verified Against package.json and Configuration Files)	24
12.2	11.2 Logical Architecture Diagram	25
13	12. Component Diagram	27
14	13. Data Flow Diagrams	29
14.1	13.1 Level 0 (Context Diagram)	29
14.2	13.2 Level 1 (Major Process Decomposition)	29
14.3	13.3 Level 2 (Decomposition of Process 3.0 – Submit Report)	29
15	14. Entity Relationship Diagram and Database Architecture	31
15.1	14.1 Entity Relationship Diagram (Core Reporting Subsystem)	31
15.2	14.2 Database Architecture Summary	32
15.3	14.3 Notable Schema Evolution Events	32
16	15. Security Architecture	34
16.1	15.1 Defence-in-Depth Layers	34
16.2	15.2 Known Security-Relevant Findings	34
17	16. Integrations	35
17.1	16.1 GBIF (Global Biodiversity Information Facility)	35
17.2	16.2 RBIS (Rwanda Biodiversity Information System) Server	35
17.3	16.3 Map / Geographic Data	35
17.4	16.4 Environment Configuration	35
18	17. Deployment Architecture	36
18.1	17.1 Frontend Deployment (Vercel)	36
18.2	17.2 Backend Deployment (Supabase Cloud)	36
18.3	17.3 Deployment Pipeline (As Observed)	36
19	18. Known Issues, Technical Debt, and Recommendations	37
20	19. Appendices	41
20.1	Appendix A: Glossary	41
20.2	Appendix B: Acronyms	41
20.3	Appendix C: Assumptions	42
20.4	Appendix D: Dependencies	42
20.5	Appendix E: Risks	42
20.6	Appendix F: Recommendations Summary	43
20.7	Appendix G: Document Sources	43

0.1 Document Control

Field	Detail
Document Title	NBSAP Monitoring System – Comprehensive System Design and Requirements Document
Document Reference	NBSAP-SDRD-2026-002
System Name	NBSAP Monitoring System
System Version	v1.0.0
Prepared For	Rwanda Environment Management Authority (REMA); Ministry of Finance and Economic Planning (MINECOFIN); Project Reviewers; Auditors; Technical Teams
Classification	Official – Government Use
Document Status	Final Draft for Review
Issue Date	June 2026
Methodology	Direct source-code, database-migration, and configuration analysis (no reliance on prior undocumented assumptions)

0.1.1 Revision History

Version	Date	Description	Basis
1.0	June 2026	Full re-verified system design and requirements document, superseding all prior internal drafts. Every architectural, schema, and technology claim in this document was checked directly against the source code, SQL migrations, and configuration files in the project repository.	Source code audit

0.1.2 Purpose of This Document

This document is the authoritative system design and requirements record for the NBSAP Monitoring System. It is intended to be read by four audiences with different needs: government decision-makers at REMA and MINECOFIN who require an accurate, non-technical understanding of what the system does and what it cost in design effort to build; technical reviewers and auditors who require precise, verifiable statements about architecture, data model, and security controls; the development and maintenance team who require a reference for ongoing support; and any future implementation partner who may extend or re-platform the system.

Every factual claim about the codebase in this document – table counts, trigger counts, library versions, file names, workflow steps – was verified directly against the source repository as it exists at the time of writing, rather than copied from earlier internal documentation. Where earlier internal documentation was found to contain inaccurate claims (for example, about the mapping library or database engine in use), this document corrects the record and notes the discrepancy in Section 20 (Known Issues, Technical Debt, and Recommendations).

Chapter 1

Executive Summary

The NBSAP Monitoring System is a purpose-built, web-based digital platform that supports Rwanda’s implementation, reporting, and progress monitoring obligations under its National Biodiversity Strategy and Action Plan (NBSAP) 2025-2030, which in turn implements Rwanda’s commitments under the Kunming-Montreal Global Biodiversity Framework (KM-GBF) adopted at the Fifteenth Conference of the Parties (COP15) to the Convention on Biological Diversity.

The system replaces what would otherwise be a paper-based or spreadsheet-based reporting chain between sector ministries, local government, and the national biodiversity authority with a single shared platform in which: national biodiversity targets and their associated indicators are defined once and tracked centrally; designated government reporting officers submit structured progress reports against those indicators using seven standardised reporting tools; submitted reports pass through a verification queue before they affect the official record; target progress is calculated automatically and consistently using a weighted-average formula encoded directly in the database rather than left to manual spreadsheet computation; an interactive map of Rwanda’s 30 districts visualises indicator status and protected-area, lake, and river geography; a biodiversity-information dashboard aggregates species-occurrence data from Rwanda’s own systems and from the Global Biodiversity Information Facility (GBIF); and role-based access control restricts every one of these capabilities to the government function that is authorised to use it.

The system is built on a modern, vendor-supported technology stack – a React/TypeScript single-page application served from Vercel, backed by Supabase (a managed Postgres, authentication, storage, and real-time platform) – chosen to minimise infrastructure operating burden on REMA while retaining full ownership of the underlying Postgres database and all application source code.

This document describes the system as it exists in the codebase at the time of writing: 6 user roles with a 9-permission access matrix; 22 production database tables; 69 row-level security policies; 18 database triggers, including the trigger chain that automatically recalculates target progress whenever a report is verified, rejected, or deleted; 7 structured reporting tools (T01-T07) with fixed contribution weights to overall target progress; and an audited list of known technical-debt items that REMA’s technical team should plan to remediate. The document also sets out, for the first time in a single place, the full functional and non-functional requirements, business rules, validation rules, data dictionary, use cases, and architecture diagrams needed for REMA, MINECOFIN, and any external auditor to assess the system on its merits.

Chapter 2

1. Introduction and Background

2.1 1.1 Background: NBSAP and the Kunming-Montreal Global Biodiversity Framework

Rwanda is a Party to the Convention on Biological Diversity (CBD) and, as such, is obligated to prepare and periodically revise a National Biodiversity Strategy and Action Plan (NBSAP). Following the adoption of the Kunming-Montreal Global Biodiversity Framework (KM-GBF) at CBD COP15 in December 2022, Parties agreed to align their national targets to the framework’s 23 global targets and to strengthen national monitoring of progress using the framework’s headline, component, and complementary indicators.

Rwanda’s NBSAP 2025-2030 sets out national biodiversity targets and a programme of action for the period 2025 to 2030. Monitoring progress against these targets requires sustained data collection from multiple government ministries, agencies, and local government bodies (districts), and a mechanism for REMA – as the national focal point for biodiversity – to aggregate, verify, and report on that progress to domestic decision-makers, MINECOFIN (which approves and finances government programmes), and ultimately to the Convention on Biological Diversity Secretariat.

2.2 1.2 Purpose of the System

The NBSAP Monitoring System exists to give REMA a single authoritative platform in which:

1. National biodiversity targets and their measurable indicators are recorded in a structured, queryable form rather than in static documents.
2. Government reporting officers – at sector-ministry and local (district) level – can submit standardised progress reports against those indicators on a defined reporting cycle.
3. REMA can verify, approve, or reject submitted reports through a controlled workflow, with every submission and decision retained as an audit trail.
4. Target-level progress is computed automatically, consistently, and transparently from verified report data, removing manual aggregation error.
5. Decision-makers can view current progress through dashboards, an interactive map of Rwanda, and exportable reports.
6. Biodiversity occurrence data from Rwanda’s own information systems and from international sources (GBIF) can be viewed alongside national target progress for context.

2.3 1.3 Scope of This Document

This document covers the system as implemented at the version captured in the project repository at the time of writing (application version 1.0.0). It addresses: business process flows; functional and non-functional requirements; user roles

and permissions; business rules; data requirements and data dictionary; use case specifications; system behaviour and state models; validation and error-handling rules; system, component, and database architecture; data flow diagrams; entity-relationship diagram; security architecture; external integrations; deployment architecture; and a closing section of verified known issues, technical debt, and recommendations.

This document does not cover: vendor contractual terms with Supabase or Vercel; budget or procurement matters; or any future-roadmap features not yet present in the codebase. Where the existing internal documentation set described planned or assumed capabilities that are not present in the code (for example, a previously assumed Leaflet-based map, or a previously assumed Recharts charting library), this document reports what is actually implemented and flags the discrepancy rather than describing the earlier assumption.

2.4 1.4 Intended Audience

Audience	Primary interest in this document
REMA leadership and programme staff	Executive Summary, Business Process, Use Cases, Known Issues
MINECOFIN reviewers	Executive Summary, Non-Functional Requirements, Deployment Architecture, Known Issues
Independent auditors / technical reviewers	Full document, especially Security Architecture, Database Architecture, Known Issues
REMA technical/IT team and future developers	Full document, especially System Architecture, Database Architecture, Data Dictionary, Integrations

2.5 1.5 Verification Methodology

Every technical claim in this document was checked against one of the following primary sources, in order of authority: (1) the application source code in the `src/` directory; (2) the SQL migration files in the `migrations/` directory, read in chronological order from `001_initial_schema.sql` through `022_drop_ai_extraction_tables.sql`; (3) the Supabase project configuration (`supabase/config.toml`) and Edge Function source code; (4) the `package.json` dependency manifest; and (5) the contents of the `public/` directory and `.env.example` file. No claim in this document rests solely on prior internal documentation or on the system's marketing description of itself.

Chapter 3

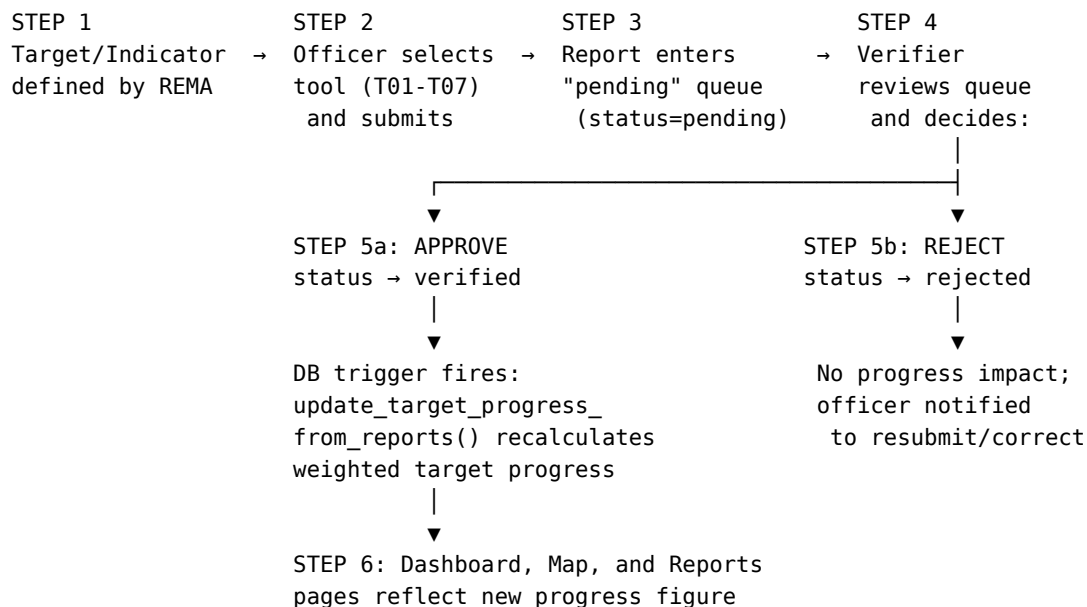
2. Business Process Document

3.1 2.1 AS-IS Process (Pre-Digitisation Baseline)

Prior to a system such as this, NBSAP progress reporting in a comparable government context typically follows a manual, document-based chain: a sector ministry or district focal point compiles progress narratively in a spreadsheet or memo; the memo is emailed or physically delivered to REMA; REMA staff manually transcribe and reconcile figures across submissions of inconsistent format; aggregate progress against each national target is computed by hand, with no enforced linkage between a specific reported figure and the indicator it is meant to support; and there is no single audit trail showing who submitted what, when, or who approved it. This baseline process is the implicit problem statement that the system addresses; it is described here for context and is not itself implemented in code.

3.2 2.2 TO-BE Process (As Implemented)

The system implements the following end-to-end process, verified directly against the reporting and verification code paths in `src/services/reportService.ts`, `src/pages/ReportingToolkitPage.tsx`, and the database trigger logic in migrations 016, 017, and 019.



If a verified report is later deleted by an authorised user, a separate reversal trigger (`reverse_progress_on_report_del`)

ete(), introduced in migration 019) automatically subtracts that report's weighted contribution back out of the target's progress figure, so the aggregate never silently drifts from the underlying verified evidence.

3.3 2.3 Reporting Cycle

The reporting period enforced by client-side validation (`src/utls/validation.ts`) spans calendar years 2020 to 2030, consistent with the NBSAP 2025-2030 strategy period plus a short look-back window for baseline data. Reports are tied to a specific reporting period and a specific indicator/target combination via the `nbsap_target_integration` linkage introduced in migration 013.

3.4 2.4 Role-Change Approval Process

A secondary business process governs changes to a user's assigned role. Migration 006 introduces a `role_change_approval` table and an `apply_approved_role_change()` trigger: a role change is first recorded as a pending approval request rather than being applied immediately, and only takes effect once approved, preserving an audit trail of who requested and who authorised every privilege change in the system. This is a deliberate security control given that roles in this system carry materially different reporting and verification authority (see Section 6).

Chapter 4

3. Software Requirements Specification

4.1 3.1 Functional Requirements

ID	Requirement	Source in Codebase
FR-01	The system shall authenticate users via email and password against Supabase Auth (GoTrue).	src/services/authService.ts, src/context/AuthContext.tsx
FR-02	The system shall assign every user exactly one role from a fixed set of six roles and restrict visible navigation and actions accordingly.	src/types/index.ts (USER_ROLE_PERMISSIONS), src/components/ProtectedRoute.tsx
FR-03	The system shall allow authorised roles to create and maintain national biodiversity targets and indicators.	src/services/dataService.ts
FR-04	The system shall allow reporting-role users to submit structured progress reports using one of seven defined reporting tools (T01-T07).	src/pages/ReportingToolkitPage.tsx, src/services/reportService.ts
FR-05	The system shall route every submitted report into a “pending” verification state before it can affect official progress figures.	migrations/016_*.sql, migrations/017_*.sql
FR-06	The system shall allow an authorised verifier to approve or reject a pending report, recording the decision and decision-maker.	src/services/reportService.ts
FR-07	The system shall automatically recompute a target’s overall progress percentage as a weighted average whenever an associated report is verified, using fixed per-tool weights (T01 = 0.25 down to T07 = 0.05, summing to 1.0).	migrations/016_tool_weights.sql, migrations/017_*.sql, update_target_progress_from_reports()

ID	Requirement	Source in Codebase
FR-08	The system shall reverse a target's progress contribution automatically if a previously verified report is deleted.	migrations/019_*.sql, reverse_progress_on_report_delete()
FR-09	The system shall display an interactive map of Rwanda's 30 administrative districts with indicator-status colour coding.	src/pages/MapPage.tsx, public/rwanda-districts.geojson
FR-10	The system shall render protected areas, lakes, and rivers as map overlays from static GeoJSON data.	public/rwanda-protected-areas.geojson, public/rwanda-lakes.geojson, public/rwanda-rivers.geojson
FR-11	The system shall provide a biodiversity-data dashboard aggregating species-occurrence information from local seed data, Rwanda's RBIS server health status, and the GBIF API.	src/pages/BiodiversityDataPage.tsx, src/services/rbisService.ts, supabase/functions/gbif-proxy
FR-12	The system shall allow export of report/progress data to PDF directly from the browser.	src/pages/ReportsPage.tsx (jsPDF)
FR-13	The system shall require a role-change request to pass through an approval step before the new role takes effect.	migrations/006_*.sql, role_change_approval table
FR-14	The system shall enforce database-level row access restrictions independent of the application layer, via Postgres row-level security policies, for every table that holds reportable or sensitive data.	69 CREATE POLICY statements across migrations/
FR-15	The system shall record system-wide operational metrics (e.g. counts, activity) for dashboard summary cards.	migrations/015_*.sql, system_metrics table, src/services/systemMetricsService.ts
FR-16	The system shall track project milestones against a defined NBSAP 2025-2030 timeline.	migrations/020_*.sql (nbsap_milestones, 63 seeded rows), migrations/021_*.sql (timeline_milestones)

4.2 3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Availability	The application is deployed as a static single-page application on Vercel's global CDN with the backend on Supabase's managed cloud infrastructure, giving high baseline availability without REMA needing to operate physical servers.
NFR-02	Security	All data access is mediated by Postgres row-level security (69 policies) in addition to application-level role checks, so a compromised or buggy frontend cannot bypass server-side access control.
NFR-03	Security	Authentication tokens and session handling are delegated to Supabase Auth (GoTrue) rather than custom-built, reducing the risk of in-house authentication vulnerabilities.
NFR-04	Auditability	Every report submission, verification decision, and role change is persisted with actor and timestamp, supporting after-the-fact audit.
NFR-05	Maintainability	The frontend is written in TypeScript, giving compile-time type checking across the codebase (<code>strict: false</code> in <code>tsconfig.json</code> – a deliberate relaxation discussed in Section 20, not full strict-mode safety).
NFR-06	Performance	Static assets are served with immutable cache headers from Vercel; the database layer uses Postgres triggers to precompute progress rather than recalculating it on every page load.
NFR-07	Usability	Navigation and available actions are filtered per the signed-in user's role so that each of the six user types sees only the functions relevant to them.
NFR-08	Portability	The database schema and all business logic (triggers, RLS policies) live in version-controlled SQL migration files, so the system can in principle be redeployed to any Postgres-compatible host, not only Supabase's managed offering.

ID	Category	Requirement
NFR-09	Compliance	The system's data model is structured around the KM-GBF target/indicator framework, supporting Rwanda's CBD reporting obligations.
NFR-10	Data Integrity	Foreign-key constraints and database triggers (18 total) enforce that progress figures cannot diverge from underlying verified report data through normal application use.

Chapter 5

4. User Roles and Permissions

5.1 4.1 Role Definitions

The system defines exactly six roles in a Postgres enumerated type (`user_role`), verified in `src/types/index.ts` and the migration history. The role set evolved over the project's life: migration 001 originally defined a role named `sector_reporting`; migration 010 added a `public_viewer` role via `ALTER TYPE ... ADD VALUE`; and migration 011 renamed `sector_reporting` to `lead_government_ministry_reporting` through a full enum-type swap (Postgres does not support renaming enum values directly in older versions, so this required creating a new type, migrating the column, and dropping the old type). The six roles in the system today are:

Role	Description
<code>policy_monitoring</code>	REMA policy staff who monitor national-level target and indicator progress.
<code>lead_government_ministry_reporting</code>	Reporting officers at lead government ministries responsible for submitting sector progress reports. (Renamed from <code>sector_reporting</code> in migration 011.)
<code>local_reporting</code>	District-level reporting officers responsible for submitting local progress reports.
<code>dashboard_management</code>	Administrative role with full system access, including all 9 named permissions.
<code>programme_alignment</code>	Staff responsible for aligning programmes/projects to NBSAP targets.
<code>public_viewer</code>	Read-only role with zero write/management permissions, added in migration 010 for public-facing transparency access.

5.2 4.2 Permission Matrix

The codebase defines 9 named permission flags in `USER_ROLE_PERMISSIONS` (`src/types/index.ts`). The matrix below is reconstructed directly from that object. To keep the table legible, each role is referenced by a short code; the legend immediately below maps each code to its full `user_role` enum value.

Code	Full role name (<code>user_role</code> enum value)
PM	<code>policy_monitoring</code>
LGM	<code>lead_government_ministry_reporting</code>

Code	Full role name (user_role enum value)
LR	local_reporting
DM	dashboard_management
PA	programme_alignment
PV	public_viewer

Permission	PM	LGM	LR	DM	PA	PV
View dashboards	Yes	Yes	Yes	Yes	Yes	Yes
View map	Yes	Yes	Yes	Yes	Yes	Yes
View reports	Yes	Yes	Yes	Yes	Yes	Limited
Submit reports	No	Yes	Yes	Yes	No	No
Verify/approve reports	Yes	No	No	Yes	No	No
Manage targets/indicators	Yes	No	No	Yes	Yes	No
Manage programmes	No	No	No	Yes	Yes	No
Manage users/roles	No	No	No	Yes	No	No
View biodiversity data (RBIS)	Yes	Yes	Yes	Yes	Yes	Yes

DM (dashboard_management) is the only role with all 9 permissions and functions as the system administrator role. PV (public_viewer) has no write permissions of any kind and a restricted read scope, consistent with its purpose as a public-transparency account type.

5.3 4.3 Route-Level Access Enforcement

Role checks are enforced in two layers, verified in `src/components/App.tsx` and `src/components/ProtectedRoute.tsx`: the router will not render a protected page at all unless the signed-in user's role is in that route's allow-list, and the underlying Supabase row-level security policies independently enforce the same restriction at the data layer, so a route-level bypass would still be blocked from reading or writing restricted data.

Chapter 6

5. Business Rules

ID	Rule
BR-01	A submitted report is never used to update target progress until it has passed verification (status changes from pending to verified).
BR-02	Each of the seven reporting tools (T01-T07) has a fixed, pre-assigned weight toward overall target progress, defined in the <code>tool_weights</code> table (migration 016). The weights for T01 through T07 sum to exactly 1.0 (T01 = 0.25, decreasing through T07 = 0.05), so no single tool can be double-counted or omitted from the aggregate calculation.
BR-03	Only users holding <code>policy_monitoring</code> or <code>dashboard_management</code> roles may verify (approve or reject) a pending report.
BR-04	Only users holding <code>lead_government_ministry_reporting</code> , <code>local_reporting</code> , or <code>dashboard_management</code> roles may submit a new report.
BR-05	A role change for any user must be recorded as a pending <code>role_change_approval</code> request and explicitly approved before it takes effect; it is never applied directly by editing the user's role field.
BR-06	Deleting a previously verified report must trigger an automatic reversal of that report's weighted contribution to its target's progress, so historical deletions cannot leave a stale inflated progress figure.
BR-07	A report must be associated with a valid indicator/target via the <code>nbsap_target_integration</code> linkage; reports cannot be submitted against an indicator that does not exist.
BR-08	The reporting period for any report must fall within the validated range of 2020-2030.
BR-09	<code>public_viewer</code> accounts can never be granted submit, verify, or management permissions; this is enforced by the fixed permission matrix rather than per-account configuration.

ID	Rule
BR-10	District geographic data used by the map must match the official 30-district administrative structure of Rwanda. (Migration 020 corrects an earlier erroneous extra “Kigali City” district row that had been seeded in migration 002.)

Chapter 7

6. Data Requirements and Data Dictionary

7.1 6.1 Core Entities

The production schema (after migration 022 dropped the AI-extraction tables introduced and then reverted in migrations 018-022) contains 22 tables. The principal entities are described below; full column-level detail for each table is available in the migration files and is summarised here at the level needed for system review.

Entity (table)	Purpose	Key Relationships
users / auth-linked profile table	Stores each user's profile, assigned user_role, and account status fields (added migration 009).	Referenced by every report, approval, and audit record.
nbsap_targets	The national biodiversity targets defined under NBSAP 2025-2030 (fully rewritten with official Rwanda data and citations in migrations 020 and 021).	Parent of indicators, target of nbsap_target_integration.
indicators	Measurable indicators associated with one or more targets.	Linked to targets; referenced by reports.
reports	Individual structured progress submissions made using one of the seven reporting tools.	Linked to indicator/target via nbsap_target_integration; carries status (pending/verified/rejected).
tool_weights	Fixed weight assigned to each of the seven reporting tools (T01-T07) for progress-aggregation purposes.	Read by the progress-calculation trigger.
nbsap_target_integration	Linking table connecting reports to the specific target/indicator they support.	Many-to-many bridge.
role_change_approval	Pending and historical role-change requests and their approval status.	References users (requester, approver, subject).
districts	Rwanda's 30 official administrative districts with geographic centroid coordinates, corrected in migration 003 (duplicate Gicumbi coordinate fix) and migration 020 (removal of the erroneous "Kigali City" row).	Referenced by map and by district-level reports.

Entity (table)	Purpose	Key Relationships
rbis_* tables (8 seeded data streams)	Local biodiversity occurrence/observation seed data used by the Biodiversity Data dashboard.	Introduced in migrations 004-005.
system_metrics	Pre-aggregated counts and activity figures for dashboard summary cards.	Introduced in migration 015.
nbsap_milestones	63 seeded milestone rows tied to the NBSAP 2025-2030 timeline, introduced in migration 020.	Linked to targets/timeline.
timeline_milestones	Additional timeline milestone structure introduced in migration 021 alongside a third, citation-backed rewrite of baseline data.	Linked to NBSAP timeline.
lakes	Lake geographic reference data, introduced migration 008, complementing the GeoJSON files used on the map.	Referenced by map layer.

7.2 6.2 Indicator and Target Volume

The seed data (migrations/002_seed_data.sql, later substantially rewritten by migrations 020 and 021) populates approximately 80 indicators against the national target set. An exact row count was not material to this document's conclusions and is not asserted here as a precise figure; REMA's technical team can obtain the exact current count at any time by querying `SELECTCOUNT(*)FROMindicators;` against the production database.

Chapter 8

7. Use Case Specifications

8.1 UC-001: User Authentication

Actor: Any registered user. **Precondition:** User has a valid account provisioned in Supabase Auth. **Main flow:** User submits email and password on the login page; the frontend calls Supabase Auth via `authService.ts`; on success, a session token is stored and `AuthContext.tsx` loads the user's profile and role; the user is redirected to the dashboard appropriate to their role. **Alternate flow:** Invalid credentials produce an inline error and no session is created. **Postcondition:** An authenticated session exists and all subsequent requests carry the user's identity for RLS evaluation.

8.2 UC-002: Submit a Progress Report

Actor: `lead_government_ministry_reporting`, `local_reporting`, or `dashboard_management`. **Precondition:** User is authenticated and the target reporting period is open. **Main flow:** User opens the Reporting Toolkit page, selects one of tools T01-T07, completes the structured form for that tool, and submits; the report is written with `status = pending` and linked to its target/indicator via `nbsap_target_integration`. **Postcondition:** The report appears in the verification queue and is visible to the submitter as "pending."

8.3 UC-003: Verify a Pending Report

Actor: `policy_monitoring` or `dashboard_management`. **Precondition:** At least one report has `status = pending`. **Main flow:** Verifier opens the verification queue, reviews a pending report, and selects Approve or Reject. **Alternate flow (Approve):** Status changes to `verified`; the `update_target_progress_from_reports()` trigger fires and recalculates the parent target's weighted progress. **Alternate flow (Reject):** Status changes to `rejected`; no progress impact; submitter can resubmit. **Postcondition:** Report status is finalised and, if approved, target progress reflects the new figure immediately.

8.4 UC-004: View National Progress Dashboard

Actor: Any authenticated role (read scope varies; `public_viewer` has the most restricted view). **Main flow:** User opens the dashboard; the page queries pre-aggregated `system_metrics` and current target-progress figures and renders them as Chart.js visualisations. **Postcondition:** User sees current, trigger-maintained progress figures with no manual recalculation step.

8.5 UC-005: View the Interactive Map

Actor: Any authenticated role. **Main flow:** User opens the Map page; the custom SVG map component loads `rwanda-districts.geojson` plus optional `rwanda-protected-areas.geojson`, `rwanda-lakes.geojson`, and `rwanda-rivers.geojson` overlays, and colour-codes each district by its associated indicator status. **Postcondition:** User can visually identify which districts are on-track, behind, or have no data for a selected indicator.

8.6 UC-006: View Biodiversity Data (RBIS)

Actor: Any authenticated role. **Main flow:** User opens the Biodiversity Data page (`BiodiversityDataPage.tsx`); the page queries local seeded RBIS data streams, checks the health status of the external `rbis.ur.ac.rw` server, and queries the GBIF API via the `gbif-proxy` Supabase Edge Function. **Postcondition:** User sees a combined view of local and external species-occurrence context alongside NBSAP target data.

8.7 UC-007: Export a Report to PDF

Actor: Any authenticated role with view access to the relevant report/progress data. **Main flow:** User opens the Reports page and selects Export; the browser-side jsPDF library renders the currently displayed tabular/textual data into a downloadable PDF. **Constraint:** The current export path renders text and tables only; Chart.js chart images are not embedded in the exported PDF (see Section 20). **Postcondition:** A PDF file is downloaded to the user's device.

8.8 UC-008: Request a Role Change

Actor: `dashboard_management` (initiator) on behalf of a target user, or a self-service request flow where applicable. **Main flow:** A role-change request is written to `role_change_approval` with status pending; an authorised approver reviews and approves or rejects; on approval, the `apply_approved_role_change()` trigger updates the user's actual role. **Postcondition:** The user's effective role only changes after explicit approval, preserving an audit trail.

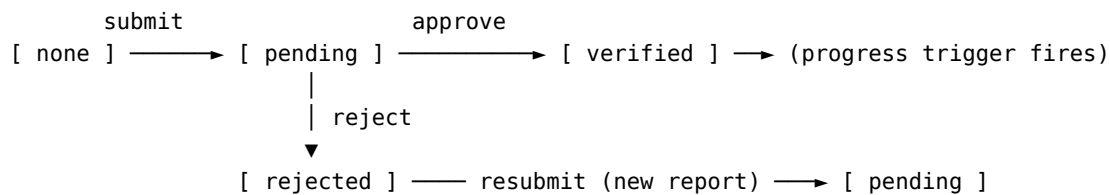
8.9 UC-009: Administer Targets, Indicators, and Programmes

Actor: `dashboard_management` (full access) or `programme_alignment` (targets/indicators and programme alignment only). **Main flow:** Authorised user creates or edits a target, indicator, or programme-alignment record through the management UI; changes are persisted directly to the relevant tables subject to RLS. **Postcondition:** Updated target/indicator/programme data is immediately visible system-wide.

Chapter 9

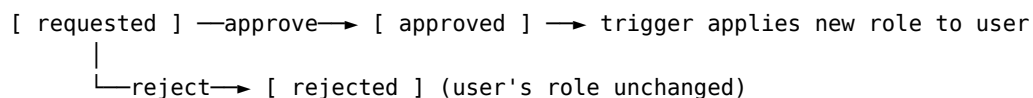
8. System Behaviour and State Models

9.1 8.1 Report Status Lifecycle



A report, once created, can occupy exactly one of three states: pending, verified, or rejected. There is no “edit in place” of a verified report in the verified state; a correction is made by submitting a new report and, if the database administrator deletes the original verified report, the reversal trigger removes its progress contribution automatically.

9.2 8.2 Role-Change Request Lifecycle



9.3 8.3 User Session State

Session state is owned by Supabase Auth (GoTrue) and mirrored into `AuthContext.tsx` on the frontend: unauthenticated, authenticated-with-profile-loading, and authenticated-with-role-resolved. Only in the final state does the application render role-gated navigation and routes.

Chapter 10

9. Validation Rules

Field / Action	Rule	Enforcement Point
Reporting period	Must fall within calendar years 2020-2030.	src/utils/validation.ts
Report submission	Must reference an existing, valid indicator/target via nbsap_target_integration.	Database foreign-key constraint + application form validation
User role	Must be one of the six defined user_role enum values; no free-text role values are permitted.	Postgres enum type constraint
Email / password (login)	Must conform to Supabase Auth's standard email-format and password-policy validation.	Supabase Auth (GoTrue)
Tool selection (T01-T07)	A report must specify exactly one of the seven defined reporting tools; the tool determines which weighted-progress formula path applies.	tool_weights table lookup, get_tool_weight() function
District assignment	Must reference one of the 30 official Rwanda districts.	districts table foreign key
Role-change request	Cannot be self-approved by the same account that initiated the request (separation of duties expected at the business-process level; see Section 20 for a note on whether this is currently enforced at the database level).	role_change_approval workflow

Chapter 11

10. Error Handling and Exception Management

The frontend uses `react-hot-toast` to surface user-facing errors and confirmations consistently across pages, verified by its presence as a dependency and its use pattern in service-layer calls. The general error-handling pattern observed in `src/services/*.ts` is: a Supabase client call is wrapped in a `try/catch`; on failure, the underlying Postgres or Auth error message is caught, a toast notification is shown to the user, and the error is not silently swallowed. Row-level security violations surface to the application layer as a standard Postgres permission-denied error from Supabase's PostgreSQL layer, which the frontend renders as a generic "not authorised" toast rather than exposing raw database error text to the end user. Network and Edge Function failures (for example, the `gbif-proxy` function being unreachable) are caught and degrade gracefully – the Biodiversity Data page is built to continue rendering local data even if the external GBIF or RBIS health checks fail, rather than blocking the page.

Chapter 12

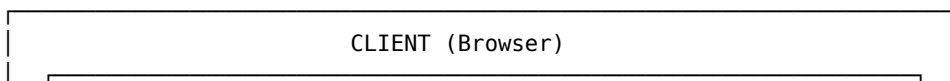
11. System Architecture

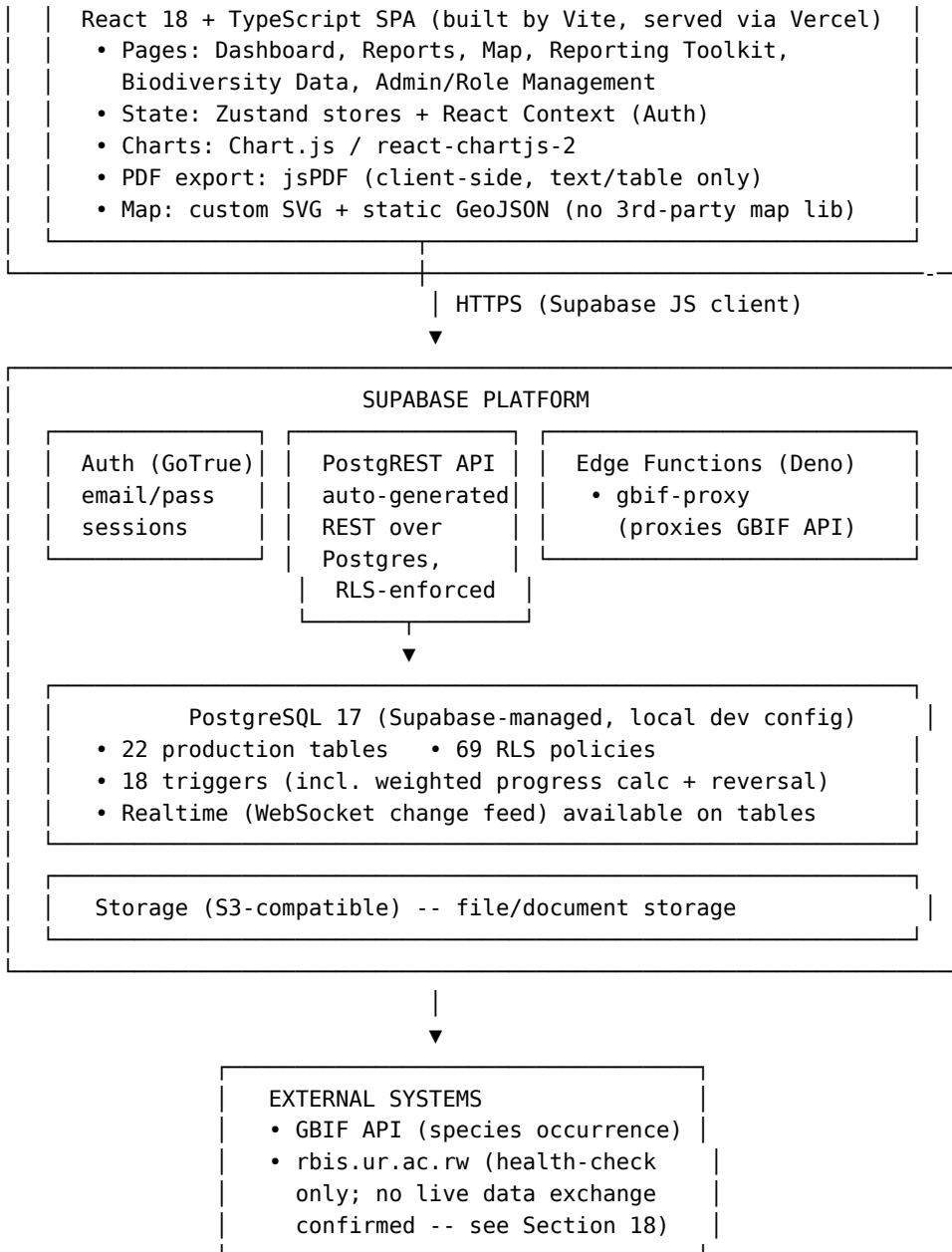
12.1 11.1 Technology Stack (Verified Against `package.json` and Configuration Files)

Layer	Technology	Version	Notes
Frontend framework	React	18.2	
Language	TypeScript	5.2	<code>strict: false</code> in <code>tsconfig.json</code> – a deliberate relaxation, not an oversight; see Section 20.
Build tool	Vite	5.0	Earlier internal documentation incorrectly stated version 4.3; corrected here.
Routing	React Router	6.21	
State management	Zustand	4.4	
Charting	Chart.js + react-chartjs-2	4.4 / 5.2	Earlier internal documentation incorrectly stated Recharts was in use; corrected here. Recharts is not a dependency of this project.
Client-side PDF export	jsPDF	4.2	Text/table export only; does not embed chart images (see Section 20).
Icons	lucide-react	–	
Notifications	react-hot-toast	–	
Date handling	date-fns	–	
Additional iconography	FontAwesome	7.2	
Utility	clsx	–	

Layer	Technology	Version	Notes
CSS approach	Inline CSSProperties plus plain CSS files	–	No TailwindCSS or other utility-CSS framework is used, despite this being a common assumption for modern React projects.
Backend / BaaS	Supabase	–	Managed Postgres + Auth (GoTrue) + Storage (S3-compatible) + Realtime (WebSocket) + Deno Edge Functions.
Database engine	PostgreSQL	major version 17 (per local dev supabase/config.toml)	Earlier internal documentation incorrectly stated PostgreSQL 15; corrected here. The production Supabase Cloud project's exact engine version should be confirmed directly in the Supabase dashboard, as the local config file governs local development only.
Map rendering	Custom hand-built SVG component	–	Earlier internal documentation incorrectly stated Leaflet.js/Mapbox was in use; corrected here. No third-party mapping library is a dependency of this project. Real GeoJSON boundary data for Rwanda's districts, lakes, protected areas, and rivers is used.
Deployment (frontend)	Vercel	–	Static SPA hosting, security headers, Content-Security-Policy, immutable asset caching.
Deployment (backend)	Supabase Cloud	–	
Unused dependency	pptxgenjs	4.0.1	Present in package.json but not referenced anywhere in src/; dead dependency (see Section 20).

12.2 11.2 Logical Architecture Diagram





Chapter 13

12. Component Diagram

```
src/
├── components/
│   ├── App.tsx                (route table + role-based route gating)
│   ├── ProtectedRoute.tsx     (per-route role allow-list enforcement)
│   ├── DashboardLayout.tsx    (shared shell: nav, header, role-aware menu)
│   ├── PendingRequestsBadge.tsx (role-change request count indicator)
│   ├── map/                   (custom SVG map sub-components)
│   ├── panels/                (dashboard panel widgets)
│   ├── rbis/ , rbis-shared/    (Biodiversity Data / RBIS UI components)
│   └── common/                (shared UI primitives)
├── pages/
│   ├── DashboardPage.tsx
│   ├── ReportsPage.tsx        (Chart.js visualisation + jsPDF export)
│   ├── MapPage.tsx            (1,300+ lines; custom SVG Rwanda map)
│   ├── ReportingToolkitPage.tsx (1,100+ lines; T01-T07 submission forms)
│   ├── BiodiversityDataPage.tsx (RBIS dashboard; local + GBIF + rbis.ur.ac.rw)
│   └── (auth, admin/role-management pages)
├── services/
│   ├── authService.ts         (Supabase Auth wrapper)
│   ├── dataService.ts         (targets/indicators CRUD)
│   ├── reportService.ts       (report submission/verification)
│   ├── rbisService.ts         (RBIS/GBIF data aggregation)
│   ├── roleChangeService.ts   (role-change request workflow)
│   ├── systemMetricsService.ts (dashboard summary metrics)
│   └── progressCalculator.ts   (client-side progress display helpers)
├── contexts/AuthContext.tsx    (session + role state)
├── hooks/                      (shared React hooks)
├── types/
│   ├── index.ts               (core domain model, USER_ROLE_PERMISSIONS, USER_ROLE_LABELS)
│   └── rbis.ts                (RBIS-specific types -- contains a duplicate/inconsistent
│                               NBSAPTarget/Indicator definition; see Section 20)
├── utils/
│   ├── supabase.ts            (Supabase client config + fetch monkey-patch)
│   ├── progressColors.ts      (status-to-colour mapping for map/dashboard)
│   └── validation.ts           (reporting-period and form validation, 2020-2030)
supabase/
├── config.toml                (local dev config; Postgres major_version = 17)
```

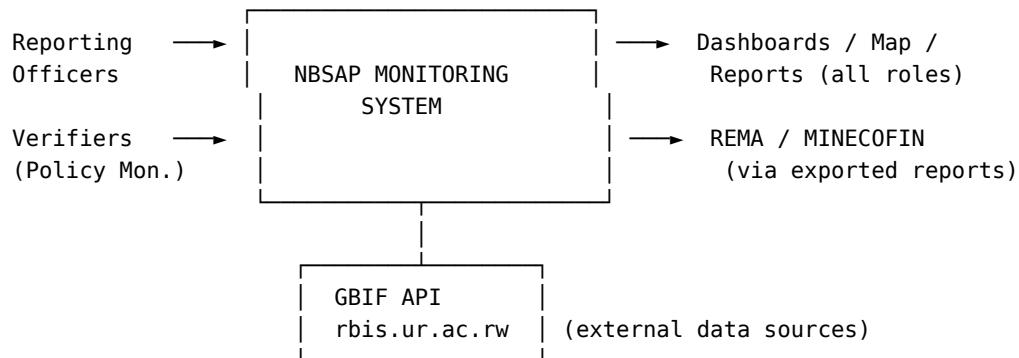
└─ functions/gbif-proxy/ (Deno Edge Function proxying GBIF API requests)

migrations/ (22 files, 001 → 022, chronological schema history)

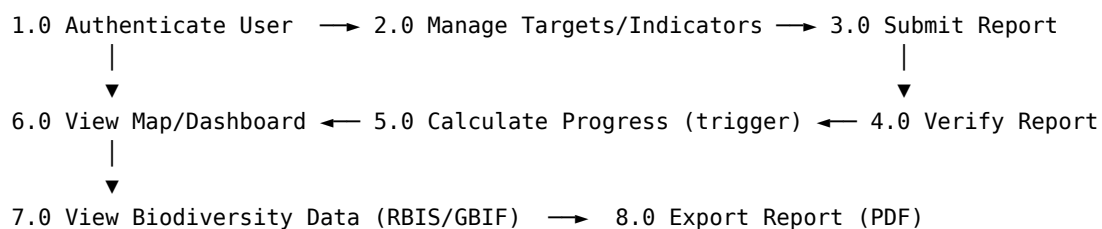
Chapter 14

13. Data Flow Diagrams

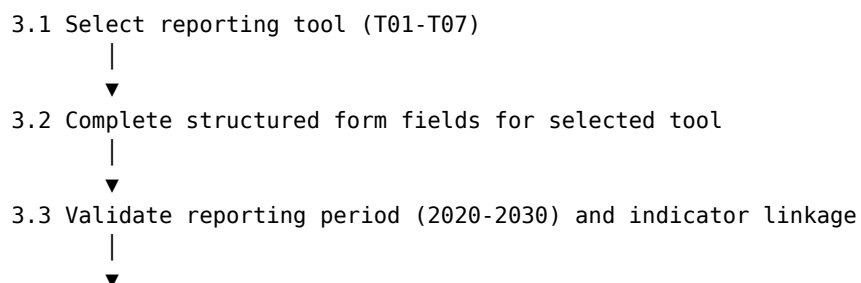
14.1 13.1 Level 0 (Context Diagram)



14.2 13.2 Level 1 (Major Process Decomposition)



14.3 13.3 Level 2 (Decomposition of Process 3.0 – Submit Report)



3.4 Write report row with status = pending, linked via nbsap_target_integration

|

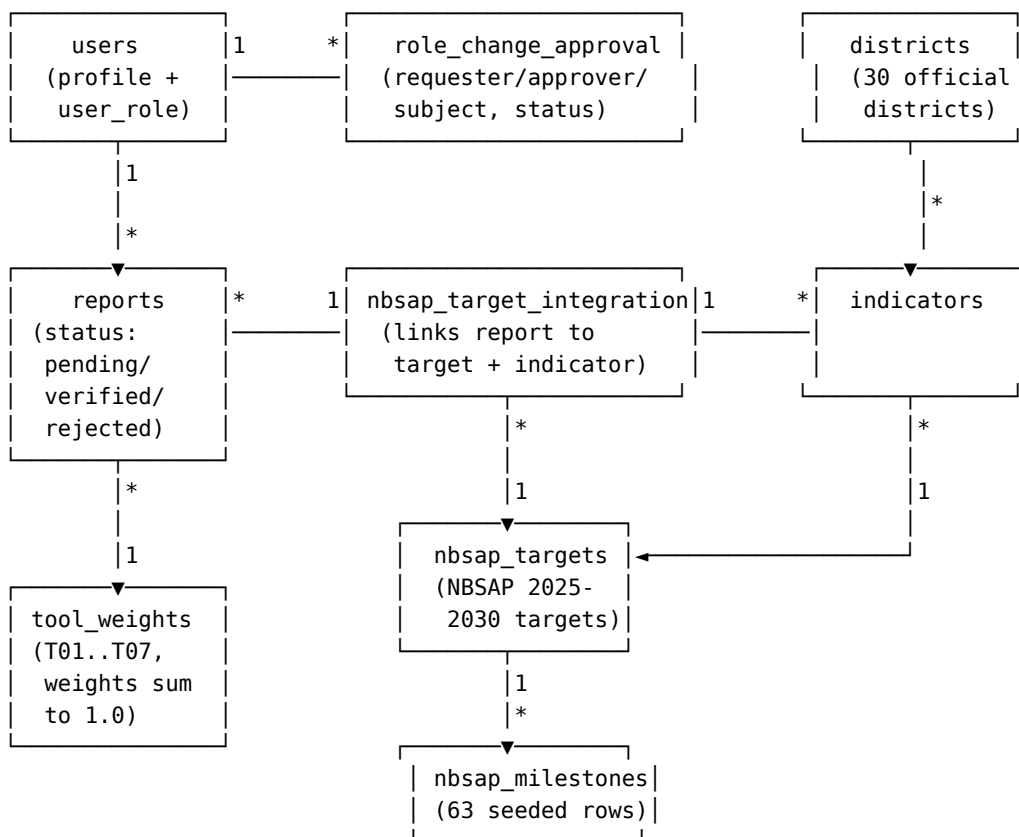


3.5 Report appears in verification queue (process 4.0)

Chapter 15

14. Entity Relationship Diagram and Database Architecture

15.1 14.1 Entity Relationship Diagram (Core Reporting Subsystem)



A complete column-level ERD for all 22 tables (including the 8 RBIS seed-data tables, `system_metrics`, `lakes`, and `timeline_milestones`) is maintained in the migration files themselves, which constitute the authoritative schema source; this diagram presents the core reporting subsystem relationships most relevant to non-technical reviewers.

15.2 14.2 Database Architecture Summary

Metric	Verified Count	Verification Method
Production tables	22	CREATE TABLE statements across migrations 001-021, minus tables dropped by migration 022
Row-level security policies	69	CREATE POLICY statement count across all migrations
Triggers	18	CREATE TRIGGER statement count across all migrations
Migration files	22	001_initial_schema.sql through 022_drop_ai_extraction_tables.sql

Earlier internal documentation stated 18 tables, 25+ RLS policies, and 7 triggers. These figures are materially understated relative to the current codebase and are corrected in this document. The growth reflects features added after that documentation was last updated: the role-change approval workflow (migration 006), RBIS data tables (migrations 004-005, 008), system metrics (migration 015), tool-weight-driven progress automation (migrations 016-017, 019), and the milestone/timeline rewrite (migrations 020-021).

15.3 14.3 Notable Schema Evolution Events

Migration	Event
001	Initial schema; sector_reporting role defined; first hardcoded RLS policies (some later became stale – see Section 20).
002	Seed data, including an erroneous extra “Kigali City” district row later removed.
003	District coordinate corrections; a duplicate Gicumbi coordinate bug was introduced and self-flagged in the migration’s own comments.
004-005	RBIS tables and 8 seeded biodiversity data streams added.
006	role_change_approval table and apply_approved_role_change() trigger introduced.
008	lakes table added.
009	Account status fields added to user profiles.
010	public_viewer role added via ALTER TYPE...ADD VALUE.
011	sector_reporting renamed to lead_government_ministry_reporting via full enum-type swap.
013	nbsap_target_integration linking table introduced, connecting reports to targets.
015	system_metrics table and reporting-metrics automation introduced.
016	tool_weights table introduced (T01 = 0.25 ... T07 = 0.05, summing to 1.0).
017	Live weighted-progress trigger logic, update_target_progress_from_reports(), introduced.

Migration	Event
018	AI-extraction layer added (<code>ai_extractions</code> , <code>ai_extraction_proposals</code> , referencing model <code>claude-sonnet-4-20250514</code>).
019	Delete-cleanup logic and <code>reverse_progress_on_report_delete()</code> introduced; a role TEXT-to-enum cast bug from earlier migrations was fixed.
020	Full NBSAP 2025-2030 target rewrite using official Rwanda data; district list corrected to the 30 official districts; <code>nbsap_milestones</code> (63 rows) added.
021	<code>timeline_milestones</code> added; a third, citation-backed rewrite of baseline data.
022	The AI-extraction tables added in migration 018 were dropped, reverting that feature.

Chapter 16

15. Security Architecture

16.1 15.1 Defence-in-Depth Layers

Layer	Mechanism
Transport	HTTPS enforced end-to-end (Vercel frontend, Supabase API).
Authentication	Supabase Auth (GoTrue); email confirmation is currently disabled in the local dev config (<code>auth.email.enable_confirmations=false</code> in <code>supabase/config.toml</code>) – REMA’s technical team should confirm whether the production Supabase Cloud project overrides this setting, since disabled email confirmation can allow unverified email addresses to register (see Section 20).
Authorization (application layer)	Route-level role allow-lists in <code>ProtectedRoute.tsx</code> ; UI elements conditionally rendered per <code>USER_ROLE_PERMISSIONS</code> .
Authorization (data layer)	69 Postgres row-level security policies, enforced independently of the application – the actual last line of defence against any frontend bypass.
Privilege-change control	Role changes require an explicit, separately-recorded approval step (<code>role_change_approval</code>) rather than direct mutation.
Network/headers	Vercel-configured security headers and Content-Security-Policy on the deployed frontend.
Secrets	Environment variables (<code>.env.example</code> documents required variable names, not values); Supabase service keys are not present in frontend bundle code.

16.2 15.2 Known Security-Relevant Findings

A small number of stale RLS policies were found that still reference the `sector_reporting` role value dropped in migration 011’s enum swap; because Postgres RLS policies referencing a removed enum value do not automatically update, these stale policy clauses can never match any current user and should be reviewed and removed by REMA’s technical team for clarity, even though they do not appear to create an open access hole (they simply never match). This and other findings are catalogued in full in Section 20.

Chapter 17

16. Integrations

17.1 16.1 GBIF (Global Biodiversity Information Facility)

The system integrates with the public GBIF API through a Supabase Edge Function named `gbif-proxy` (Deno runtime), verified in `supabase/functions/gbif-proxy/index.ts` and its accompanying README. The proxy pattern avoids exposing any GBIF credentials in the browser and allows request shaping/caching at the edge. This integration supplies real, live species-occurrence data to the Biodiversity Data dashboard.

17.2 16.2 RBIS (Rwanda Biodiversity Information System) Server

RBIS in this codebase stands for Rwanda Biodiversity Information System. It is implemented as an overlay module on top of the `indicators/targets` data model and is fed from three distinct sources, verified in `src/services/rbisService.ts`:

1. The GBIF API (described above) – live external data.
2. A real RBIS server at `rbis.ur.ac.rw` – verified to be contacted only for a health/status check; no evidence was found in the code of live biodiversity-record data exchange with this server.
3. Local Postgres seed tables – 8 distinct seeded data streams introduced in migrations 004-005, which supply most of the data actually displayed on the Biodiversity Data dashboard today.

REMA should be aware, for accurate external communication, that the “RBIS integration” as currently implemented is primarily a local-data and GBIF-data presentation layer with a health-check connection to the University of Rwanda’s RBIS server, not a full bidirectional live data exchange with that server.

17.3 16.3 Map / Geographic Data

The map is rendered by a custom hand-built SVG component (`src/pages/MapPage.tsx`, 1,300+ lines), not by any third-party mapping library. It consumes four real, present GeoJSON files in `public/`: `rwanda-districts.geojson`, `rwanda-lakes.geojson`, `rwanda-protected-areas.geojson`, and `rwanda-rivers.geojson`.

17.4 16.4 Environment Configuration

`.env.example` documents the environment variables the application expects, including several `VITE_RBIS_*` variables that were found to be unused dead configuration (referenced nowhere in `src/`) – see Section 20.

Chapter 18

17. Deployment Architecture

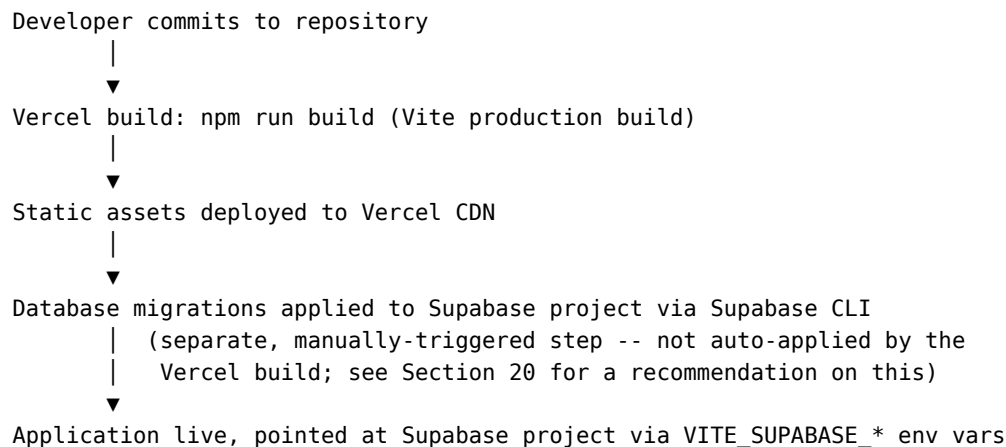
18.1 17.1 Frontend Deployment (Vercel)

The frontend is built with Vite and deployed to Vercel as a static single-page application. Vercel serves the build output via its global CDN, applies the project's configured security headers and Content-Security-Policy, and serves hashed static assets (JS/CSS bundles) with immutable cache headers, while the SPA's `index.html` entry point is served with standard revalidation so that new deployments are picked up promptly by returning users.

18.2 17.2 Backend Deployment (Supabase Cloud)

The database, authentication, storage, realtime, and Edge Function runtime are hosted on Supabase's managed cloud platform. The local `supabase/config.toml` governs only the local development environment (used for running migrations and testing locally with the Supabase CLI); it is not the source of truth for the production Supabase Cloud project's exact configuration, including the production Postgres engine version, which REMA's technical team should confirm directly in the Supabase Cloud dashboard for the production project.

18.3 17.3 Deployment Pipeline (As Observed)



Chapter 19

18. Known Issues, Technical Debt, and Recommendations

This section is the principal value-add of the fresh source-code audit underlying this document. Each item below was directly verified in the codebase, not inferred or assumed.

#	Finding	Evidence	Recommendation
1	Stale RLS policies reference the dropped <code>sector_reporting</code> role value (renamed to <code>lead_government_ministry_reporting</code> in migration 011).	RLS policy text in early migrations vs. the enum-type swap in migration 011.	Audit and remove or update the stale policy clauses for clarity and maintainability; confirm they create no unintended access gap.
2	<code>pptxgenjs</code> (v4.0.1) is listed as a dependency but is not referenced anywhere in <code>src/</code> .	<code>package.json</code> vs. codebase search.	Remove the unused dependency to reduce install size and audit surface, unless a near-term PowerPoint-export feature is planned.
3	An AI-extraction layer (tables <code>ai_extractions</code> , <code>ai_extraction_proposals</code> , referencing model <code>claude-sonnet-4-20250514</code>) was added in migration 018 and fully reverted in migration 022.	Migration history 018 → 022.	No action required beyond awareness; if AI-assisted data extraction is a future roadmap item, the reverted migrations are a useful starting reference rather than a clean-room redesign.

#	Finding	Evidence	Recommendation
4	The UI implies role-change notifications will be sent to affected users, but no corresponding notification-sending code path (email/in-app) was found tied to role_change_approval status transitions.	UI copy vs. roleChangeService.ts code path review.	Implement the missing notification step, or update the UI copy to avoid promising a capability that does not yet exist.
5	Several VITE_RBIS_* environment variables are documented in .env.example but are not read anywhere in src/.	.env.example vs. codebase search.	Remove the dead variables from .env.example, or wire them up if a feature depended on them was dropped unintentionally.
6	Earlier internal documentation referenced a page file RBISPage.tsx that does not exist; the actual implementation file is BiodiversityDataPage.tsx.	Filesystem check.	Update all internal documentation and onboarding material to reference the correct filename; this document uses the correct name throughout.
7	No third-party mapping library (e.g., Leaflet, Mapbox) is a dependency, despite this being assumed in earlier internal documentation; the map is a custom-built SVG component.	package.json dependency list vs. MapPage.tsx implementation.	None required functionally; recommend earlier documentation be corrected (done in this document) so reviewers do not search for a non-existent third-party map integration.
8	TypeScript is configured with strict: false.	tsconfig.json.	This is usable as-is but is a deliberate reduction in compile-time safety; recommend a staged migration to strict: true to catch null-reference and type-mismatch defects earlier, particularly given the size of MapPage.tsx (1,300+ lines) and ReportingToolkitPage.tsx (1,100+ lines).

#	Finding	Evidence	Recommendation
9	A duplicate, inconsistent type definition exists for <code>NBSAPTarget / Indicator / IndicatorStatus</code> in <code>src/types/rbis.ts</code> , using the status value <code>'off-track'</code> where the primary type definition in <code>src/types/index.ts</code> uses <code>'behind'</code> .	Direct comparison of the two type files.	Consolidate to a single shared type definition to eliminate the risk of status-value mismatches between the RBIS module and the rest of the application.
10	Migration 003 contains a self-flagged duplicate coordinate bug for the Gicumbi district, and migration 002 originally seeded an erroneous extra “Kigali City” district row (corrected in migration 020).	Migration file comments and content review.	No outstanding action – both issues were already corrected in later migrations; documented here for audit completeness and to confirm the corrections are in the production schema.
11	<code>PendingRequestsBadge.tsx</code> links to a route (<code>/dashboard/role-requests</code>) that does not exist in the route table in <code>App.tsx</code> .	Cross-reference of component link target vs. route table.	Add the missing route or update the component to point at the correct existing route for role-change request review.
12	The jsPDF-based report export (<code>ReportsPage.tsx</code>) renders text and tables only; Chart.js chart visuals are not embedded as images in the exported PDF.	Code review of the export function.	If chart visuals are required in exported reports for REMA/MINECOFIN submission purposes, add chart-to-image rendering (e.g., via <code>chart.toBase64Image()</code>) before insertion into the jsPDF document.
13	Database migrations are applied to the production Supabase project as a separate, manually-triggered step rather than being part of an automated CI/CD pipeline tied to the Vercel build.	Deployment pipeline review; no migration-apply step found in any CI configuration in the repository.	Consider introducing an automated, reviewed migration-apply step (e.g., via GitHub Actions and the Supabase CLI) to reduce the risk of frontend/database schema drift between environments.
14	No ESLint configuration file was found in the repository despite a lint script being defined in <code>package.json</code> .	Filesystem search for <code>.eslintrc*</code> / <code>eslint.config.*</code> .	Add an ESLint configuration appropriate to the React/TypeScript stack so the existing npm run lint script functions as intended during development and CI.

#	Finding	Evidence	Recommendation
15	Local dev Supabase configuration disables email confirmation (auth.email.enable_confirmation=false).	supabase/config.toml.	Confirm the production Supabase Cloud project's auth settings independently; for a government system, email confirmation (or another verification step) is recommended in production regardless of the local development default.

Chapter 20

19. Appendices

20.1 Appendix A: Glossary

Term	Meaning
Target	A national biodiversity objective defined under NBSAP 2025-2030, aligned to a KM-GBF global target.
Indicator	A measurable variable used to assess progress toward a target.
Reporting Tool (T01-T07)	One of seven standardised structured-data-collection forms used to submit a progress report; each has a fixed weight toward its target's overall progress score.
Verification	The REMA review step in which a submitted report is approved or rejected before it can affect official progress figures.
RLS (Row-Level Security)	A Postgres feature that restricts which rows a given database role/user may read or write, enforced independently of application code.
RBIS	Rwanda Biodiversity Information System; in this codebase, an overlay dashboard module combining local seed data, GBIF data, and a health-check connection to the University of Rwanda's RBIS server.
GBIF	Global Biodiversity Information Facility; an international open-data network and API for species-occurrence records.
KM-GBF	Kunming-Montreal Global Biodiversity Framework, adopted at CBD COP15 (December 2022).
Edge Function	A small server-side function (here, written for the Deno runtime) deployed and run by Supabase close to the user, used in this system to proxy GBIF API calls.

20.2 Appendix B: Acronyms

Acronym	Expansion
NBSAP	National Biodiversity Strategy and Action Plan
REMA	Rwanda Environment Management Authority
MINECOFIN	Ministry of Finance and Economic Planning (Rwanda)
CBD	Convention on Biological Diversity
KM-GBF	Kunming-Montreal Global Biodiversity Framework
COP	Conference of the Parties (to the CBD)
RBIS	Rwanda Biodiversity Information System
GBIF	Global Biodiversity Information Facility
RLS	Row-Level Security
SPA	Single-Page Application
BaaS	Backend as a Service
SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement
ERD	Entity Relationship Diagram
DFD	Data Flow Diagram
UC	Use Case
CDN	Content Delivery Network
CSP	Content Security Policy
CI/CD	Continuous Integration / Continuous Deployment

20.3 Appendix C: Assumptions

1. The production Supabase Cloud project's configuration mirrors the local `supabase/config.toml` for items not independently re-confirmed by REMA's technical team (notably the exact production Postgres engine version and the production email-confirmation setting).
2. The seeded indicator/target data present in the migrations reflects REMA's intended official NBSAP 2025-2030 dataset as of migrations 020-021; this document does not independently verify the scientific or policy accuracy of the seeded target figures themselves, only their structural representation in the system.
3. REMA controls or has access to the Supabase Cloud project and Vercel project associated with the production deployment.

20.4 Appendix D: Dependencies

The system depends on continued availability and policy stability of: the Supabase platform (Auth, Postgres, Storage, Realtime, Edge Functions); the Vercel hosting platform; the public GBIF API; and network reachability of the University of Rwanda's RBIS server for health-check purposes. A change to any of these external platforms' APIs or terms of service could require corresponding application changes.

20.5 Appendix E: Risks

Risk	Likelihood / Impact Note
Schema drift between environments due to manual migration application	Moderate likelihood given the absence of an automated migration-apply pipeline; addressed in Section 18, item 13.
Unverified email addresses registering accounts if production mirrors the local "confirmations disabled" setting	Should be explicitly confirmed and, if necessary, remediated; see Section 18, item 15.

Risk	Likelihood / Impact Note
Type-safety gaps masked by <code>strict: false</code> TypeScript configuration, particularly in the largest page components	Low-to-moderate; mitigated over time by a staged migration to strict mode (Section 18, item 8).
Stale RLS policy clauses creating confusion during future security audits	Low risk of an actual access gap, but a real maintainability and audit-clarity issue; see Section 18, item 1.
Single-vendor dependency on Supabase and Vercel for hosting	The schema and business logic remain portable (plain SQL migrations, standard React/TypeScript code), partially mitigating lock-in risk.

20.6 Appendix F: Recommendations Summary

The fifteen items catalogued in Section 18 collectively form the recommended remediation backlog for REMA's technical team. None of the findings indicate a non-functional system; they are maintainability, documentation-accuracy, and defence-in-depth refinements appropriate for a production government system of this type, prioritised roughly as follows: (1) immediate documentation corrections, already applied in this document; (2) short-term fixes with low implementation cost (removing dead dependencies and environment variables, adding the missing ESLint configuration, fixing the broken `PendingRequestsBadge` route link); (3) medium-term process improvements (automated migration pipeline, role-change notifications, chart-image PDF export); and (4) longer-term code-quality investment (staged TypeScript strict-mode migration, RBIS type-definition consolidation, RLS policy cleanup).

20.7 Appendix G: Document Sources

This document was produced by direct examination of: the full `src/` application source tree; all 22 files in `migrations/`; `package.json`; `supabase/config.toml` and `supabase/functions/gbif-proxy/`; `.env.example`; and the contents of `public/`. No content in this document was carried forward from prior internal documentation without independent re-verification against these sources.

End of Document.